PROJETO PI - HIGH EVOLUTION

Requisitos da disciplina Modelagem de Software e Arquitetura de Sistemas

INTEGRANTES DO PROJETO e RA'S

João Kiala Vioka Panzo	-	26029596
Julio Paulino de Souza Neto	-	26029729
Vinicius Mendonça Menes	-	26029699

São Paulo

2026

Sumário

1 INTRODUÇÃO

1.1 – Processo de desenvolvimento de Software

1.2 Apresentação

2. DESIGN SPRINT – Ideação e prototipação do desafio

2.1 Desafio

2.2 Entender Mapear

2.2.1 – Entendimento e observação. COMO PODEMOS?

2.2.2 – Pesquisa Desk

2.3 Ideação – desenho da solução (trilha do usuário)

2.4 Prototipagem

3.REQUISITOS DE SISTEMA

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE

4. DIAGRAMAS DE UML

4.1 CASOS DE USO

4.2 DIAGRAMA DE CLASSE

5. Especificação estrutura de uma rotina do sistema

5.1 – Apresentar a especificação de acordo com o modelo abaixo

5.2 – Apresentar a codificação da rotina

6. ARQUITETURA DO SISTEMA

6.1. Visão Geral do Sistema

6.2. Arquitetura Geral

6.3. Componentes Arquiteturais e Camadas

6.4. Diagrama da Arquitetura

6.5. Organização de Pastas e Módulos

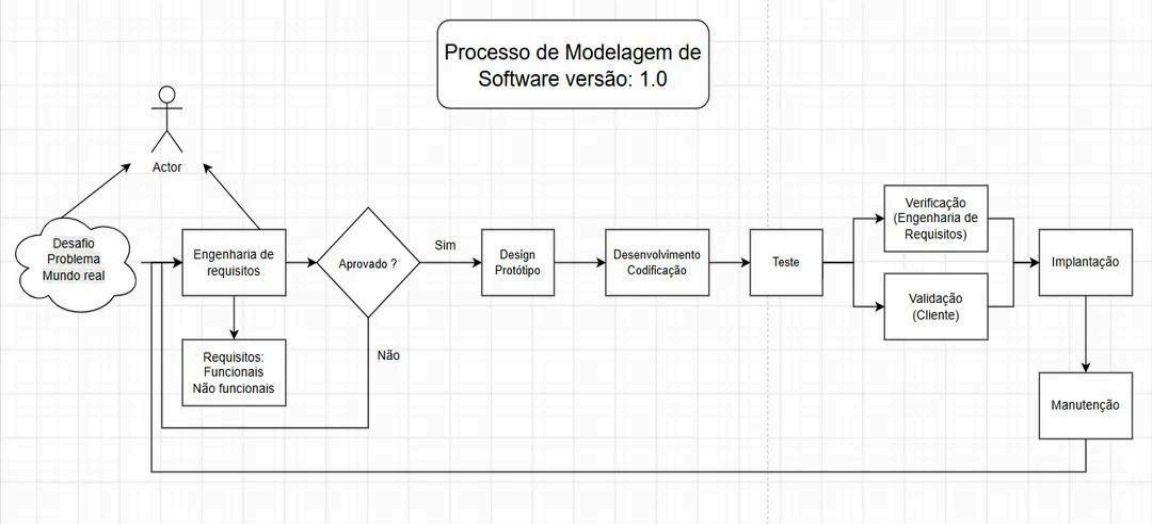
6.6. Integrações Externas (se houver)

6.7. Ambiente e Implantação

8. REFERÊNCIAS BIBLIOGRÁFICAS

1 INTRODUÇÃO

1.1 – Processo de desenvolvimento de Software



1.2 Apresentação

A Arcor (https://www.arcor.com.br/) é uma empresa do setor alimentício que possui um amplo portfólio de produtos, incluindo chocolates, biscoitos, guloseimas e outras categorias. Como parte de uma ação de marketing e relacionamento com o público, a empresa propõe às equipes do 1º semestre do curso de Análise e Desenvolvimento de Sistemas o desenvolvimento de um sistema digital de caráter lúdico e/ou educativo, com o objetivo de aproximar os usuários da marca Arcor e de suas diferentes marcas, linhas e produtos.

A proposta consiste em transformar o conhecimento sobre o portfólio da empresa em uma experiência interativa e divertida. Para isso, o participante deverá avançar por perguntas, desafios, pistas e feedbacks que apresentem marcas como Tortuguita e Triunfo, além de outras marcas e produtos que poderão ser selecionados pela equipe em conjunto com o parceiro.

Todo o conteúdo utilizado no sistema deverá ser baseado em materiais oficiais fornecidos pela Arcor e previamente validado antes da entrega final. Dessa forma, devem ser evitadas afirmações de caráter promocional, nutricional ou relacionadas à qualidade dos produtos que não estejam devidamente fundamentadas em fontes fornecidas ou aprovadas pela empresa.

Para o desenvolvimento da solução, cada equipe poderá optar por uma das abordagens propostas ou combinar diferentes elementos em uma única experiência:

- **Modo Quiz:** sequência de perguntas nos formatos de múltipla escolha, verdadeiro ou falso e associação, com sistema de pontuação, níveis de dificuldade, feedback imediato e informações complementares sobre as marcas e os produtos apresentados.

Um dos requisitos obrigatórios do projeto é que a base de perguntas, alternativas, respostas, pistas e seus respectivos vínculos com as marcas e produtos seja armazenada em um **Banco de Dados relacional**. Portanto, não será considerada válida como solução final uma aplicação que mantenha toda a base de perguntas diretamente codificada no programa.

Além disso, cada equipe deverá definir e justificar o **público-alvo** da experiência, podendo considerar, por exemplo, adolescentes, jovens adultos ou o público geral. A partir dessa definição, deverão ser adequados aspectos como linguagem, nível de dificuldade, duração das partidas e interface, garantindo que a experiência seja compatível com o perfil dos usuários selecionados.

2. DESIGN SPRINT – Ideação e prototipação do desafio

2.1 Desafio

O nosso desafio consiste em desenvolver um sistema digital lúdico e educativo, baseado no formato Quiz, que utilize perguntas e regras de jogo para promover a interação e o aprendizado dos usuários. O sistema contará com questões de múltipla escolha, verdadeiro ou falso e associação, organizadas em diferentes níveis de dificuldade, com sistema de pontuação, feedback imediato e informações complementares sobre as marcas, linhas e produtos relacionados.

As perguntas e demais informações serão armazenadas em um Banco de Dados, com o objetivo de proporcionar aos usuários maior familiarização com a empresa Arcor, suas marcas, linhas e produtos. O desenvolvimento do sistema também permitirá a integração dos conhecimentos adquiridos nas Unidades Curriculares do 1º semestre do curso de Análise e Desenvolvimento de Sistemas, aplicando conceitos de programação, banco de dados, desenvolvimento de sistemas e lógica computacional.

2.2 Entender Mapear

2.2.1 – Entendimento e observação. COMO PODEMOS?

Como podemos definir um público-alvo?

Como podemos premiar os tops do ranking?

Como podemos classificar a dificuldade de cada fase e a pontuação por pergunta?

Como podemos tornar o quiz da Arcor divertido e fácil de jogar?

Como podemos fazer o jogador querer jogar novamente depois de terminar o quiz?

Como podemos criar desafios que sejam rápidos, mas interessantes?

Como podemos adaptar o nível de dificuldade para diferentes jogadores?

Como podemos fazer o jogador sentir que está evoluindo durante o jogo?

Como podemos criar uma experiência que seja divertida tanto para crianças quanto para adultos?

Como podemos utilizar elementos de jogos famosos sem deixar de criar uma identidade própria para a Arcor?

Como podemos fazer o jogador aprender sobre a empresa enquanto joga?

Como podemos transformar informações sobre a Arcor em perguntas interessantes?

Como podemos apresentar curiosidades sobre os produtos sem parecer uma propaganda tradicional?

Como podemos tornar o ARCOR QUEST fácil de acessar pelo celular?

Como podemos criar uma interface simples para que qualquer pessoa consiga jogar?

Como podemos permitir que o jogador compartilhe sua pontuação?

Como podemos utilizar sons, animações e efeitos para deixar o quiz mais envolvente?

Como podemos fazer o jogador interagir com a Arcor mesmo depois de terminar o quiz?

Como podemos utilizar a gamificação para aumentar o interesse pelos produtos Arcor?

Como podemos criar uma experiência que faça o jogador associar a Arcor a diversão?

Como podemos fazer o ARCOR QUEST se destacar de outros quizzes de marcas?

2.2.2 – Pesquisa Desk

Identificação dos principais concorrentes da marca Arcor para validação de dados;

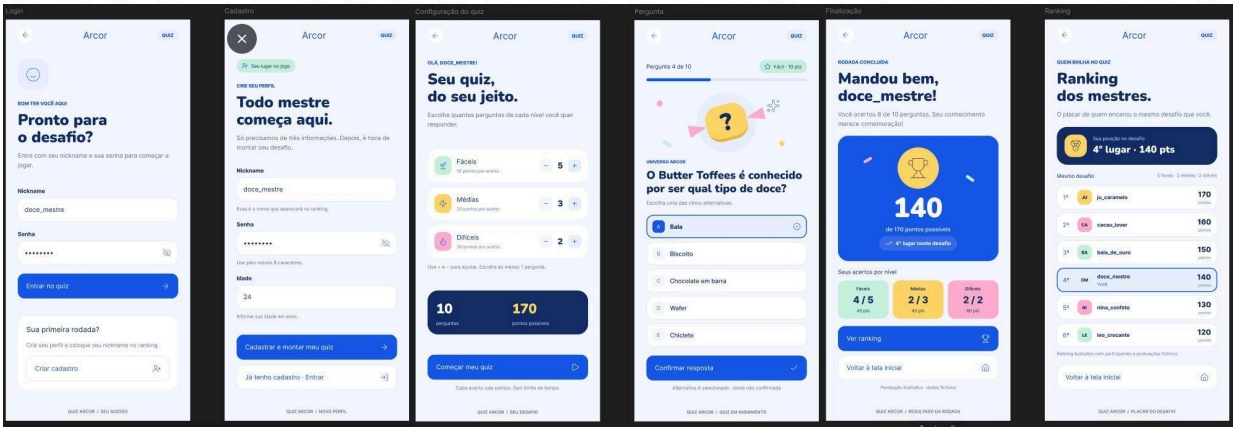
Nestle, Lacta, Milka, Sonho de Valsa, Ouro Branco, Toblerone, Garoto

2.3 Ideação – desenho da solução (trilha do usuário)

ETAPA	Cadastro	login	Tutorial	modo de jogo	pont.	ranking
O que o usuario faz	cadastra suas informações	acessa usuario e senha	aprende como jogar	seleciona a dificuldade	mostra a pontuação obtida	mostra as melhores classificações
Como ele se sente	animado	seguro	confiante/ preparado	curioso	ansioso	motivado/ recompensado
Onde ele trava	cadastro dificil	erro ao logar	tutorial complicado	muito difícil	erro de conta-bilização	obter ranking abaixo do esperado



2.4 Prototipagem



3.REQUISITOS DE SISTEMA

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE

Necessários 6 requisitos

RFS01	
Função	Cadastro e Autenticação de Usuários
Descrição	Permitir que o usuário registre seus dados iniciais de perfil e acesse a plataforma de forma segura por meio de nome de usuário e senha.
Entradas	Nome de usuário, email, senha e dados de cadastro iniciais.
Fonte	Usuário / Tela de Cadastro e Login.
Saídas	Confirmação de cadastro, token/sessão de acesso e redirecionamento para o menu principal.
Ação	O sistema valida as credenciais fornecidas, criptografa a senha, armazena os dados do usuário no banco de dados e libera o acesso à plataforma.

RFS02	
Função	Execução de Tutorial de Preparação
Descrição	Prover uma etapa de tutorial interativo para apresentar as regras e mecânicas de jogo antes do início das partidas.
Entradas	Comandos de navegação e avanço nas telas de instrução.
Fonte	Usuário / Interface do Tutorial

Saídas	Telas explicativas com dicas e exemplos visuais das marcas Arcor.
Ação	O sistema exibe o passo a passo interativo para garantir que o participante compreenda a dinâmica do quiz antes de iniciar os desafios.

RFS03	
Função	Gerenciamento de Perguntas e Respostas via Banco de Dados
Descrição	Buscar e gerenciar perguntas, alternativas, respostas, pistas e vínculos com marcas/produtos a partir de um Banco de Dados relacional, evitando código estático.
Entradas	Consultas SQL ou chamadas de API solicitando o lote de questões.
Fonte	Banco de Dados Relacional (MySQL/PostgreSQL)
Saídas	Conjunto de dados estruturados contendo as perguntas, alternativas e metadados das marcas.
Ação	O backend (Node.js + Express) consulta o banco de dados relacional para fornecer as questões dinamicamente ao frontend durante o quiz.

RFS04	
Função	Seleção de Modos de Jogo e Dificuldade
Descrição	Permitir que o usuário escolha entre diferentes categorias, marcas (ex: Tortuguita, Triunfo) e níveis de dificuldade para a realização do quiz.
Entradas	Seleção da categoria de produto e do nível de dificuldade desejado.
Fonte	Usuário / Tela de Seleção de Modos
Saídas	Inicialização da partida configurada com as questões correspondentes ao nível escolhido.
Ação	O sistema filtra e carrega as perguntas do banco de dados compatíveis com a categoria e o nível selecionados pelo jogador.

RFS05	
Função	Feedback Imediato e Pontuação em Tempo Real
Descrição	Calcular e exibir a pontuação do usuário instantaneamente após cada resposta, fornecendo feedbacks visuais de desempenho e curiosidades sobre os produtos Arcor.
Entradas	Resposta enviada pelo usuário (alternativa selecionada)
Fonte	Usuário / Interface de Jogo
Saídas	Atualização do placar na tela, mensagem de acerto/erro e exibição de informações complementares.

Ação	O sistema valida se a resposta está correta, incrementa a pontuação conforme o nível de dificuldade, atualiza o placar e apresenta a curiosidade institucional correspondente.

RFS06	
Função	Exibição de Ranking de Classificação
Descrição	Processar, ordenar e exibir um ranking com as melhores pontuações dos usuários ao final das rodadas.
Entradas	Solicitação de visualização da tabela de classificação.
Fonte	Usuário / Tela de Ranking.
Saídas	Listagem ordenada com as maiores pontuações e a posição do usuário.
Ação	O sistema consulta o banco de dados ordenando os registros de pontuação de forma decrescente e renderiza o pódio/ranking na interface do usuário.

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE

Necessários 6 requisitos

RNFS01	
Função	Usabilidade e Adequação ao Público-Alvo
Descrição	Garantir uma interface simples, intuitiva e lúdica, adequada ao perfil dos usuários (jovens e adultos) para facilitar a interação com a marca Arcor.
Entradas	Diretrizes de design, identidade visual da Arcor e perfil do público-alvo.
Fonte	Equipe de Desenvolvimento / Documento de Requisitos
Saídas	Interface responsiva e amigável baseada em componentes React.
Ação	O sistema aplica padrões de design visual limpo, navegação simplificada e elementos de gamificação para manter o engajamento do usuário.

RNFS02	
Função	Segurança de Dados
Descrição	Assegurar a proteção das credenciais de acesso dos usuários e a integridade dos dados armazenados no sistema.
Entradas	Políticas de segurança e dados sensíveis de login.
Fonte	Camada de Aplicação (Backend) / Banco de Dados

Saídas	Comunicação segura e armazenamento de senhas com criptografia
Ação	O backend aplica criptografia nas senhas e valida restrições de acesso para evitar exposição indevida de dados cadastrais.

RNFS03	
Função	Desempenho e Tempo de Resposta
Descrição	Garantir que as requisições de carregamento de perguntas e salvamento de pontuação processadas pela API em C# e consumidas pelo JavaScript retornem rapidamente para assegurar fluidez na experiência de jogo.
Entradas	Requisições HTTP/JSON enviadas pelo frontend em JavaScript para o backend em C#.
Fonte	Camada de Apresentação (Frontend JavaScript).
Saídas	Respostas da API desenvolvida em C# em tempo inferior a 2 segundos.
Ação	O servidor processa e retorna às consultas otimizadas do banco de dados através da API em C#, entregando os dados de forma ágil para o ecossistema JavaScript sem atrasos perceptíveis para o usuário.

RNFS04	
Função	Persistência Confiável
Descrição	Utilizar um sistema de gerenciamento de banco de dados relacional robusto para garantir a integridade transacional e estrutural dos dados do quiz.
Entradas	Esquema relacional de tabelas (usuários, perguntas, pontuações).
Fonte	Camada de Persistência (MySQL / PostgreSQL)
Saídas	Armazenamento seguro, consistente e recuperável de todas as informações do sistema.
Ação	O sistema gerencia transações de banco de dados assegurando que não ocorram perdas de pontuação ou corrupção de dados dos usuários.

RNFS05	
Função	Compatibilidade e Responsividade
Descrição	Assegurar que a aplicação web seja totalmente compatível com diferentes navegadores e dispositivos (smartphones e desktops).

Entradas	Resoluções de tela variadas e diferentes navegadores web.
Fonte	Dispositivos dos Usuários / Browsers
Saídas	Layout adaptável e funcional em celulares, tablets e computadores.
Ação	O frontend utiliza técnicas de design responsivo para redimensionar e reorganizar os elementos visuais conforme o tamanho da tela do dispositivo.

RNFS06	
Função	Conformidade de Conteúdo Institucional
Descrição	Garantir que todo o conteúdo informativo e curiosidades exibidas estejam rigorosamente alinhados com as fontes e diretrizes oficiais fornecidas pela Arcor.
Entradas	Materiais institucionais, manuais e portfólio oficial da empresa parceira.
Fonte	Empresa Parceira (Arcor) / Documentação do Projeto
Saídas	Textos, perguntas e dados validados institucionalmente na aplicação.
Ação	O sistema restringe a exibição de informações institucionais estritamente aos dados validados e aprovados pelas fontes oficiais da empresa .

4. DIAGRAMAS DE UML

4.1 CASOS DE USO

UC01 – Cadastrar e autenticar usuário

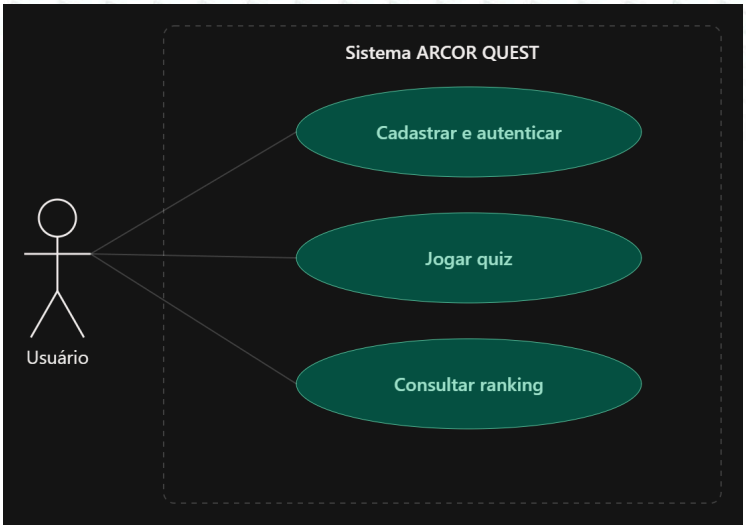
Ator: Usuário (jogador). O usuário informa login, senha e idade para se cadastrar e, em seguida, autentica-se para acessar o menu principal.

UC02 – Jogar quiz

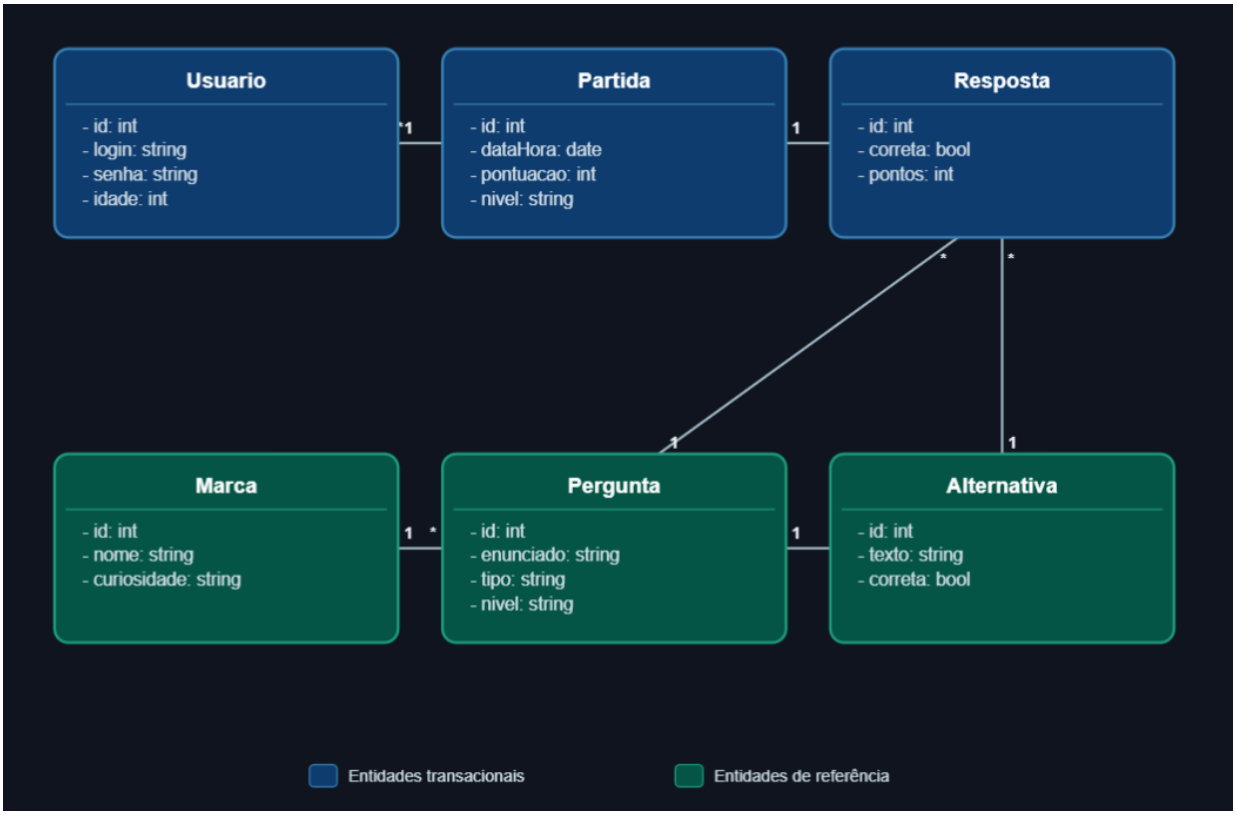
Ator: Usuário (jogador). O usuário percorre o tutorial, seleciona marca/dificuldade , responde às perguntas buscadas no banco de dados e recebe pontuação e feedback em tempo real.

UC03 – Consultar ranking

Ator: Usuário (jogador). O usuário visualiza a lista das melhores pontuações e sua posição.



4.2 DIAGRAMA DE CLASSE



5. Especificação estrutura de uma rotina do sistema

5.1 – Apresentar a especificação de acordo com o modelo abaixo

Sistema/Software

Função

Descrição

Entradas

Fonte

Saídas

Destino

Ação

Requisitos

Pré-condição

Pós-condições

Efeitos colaterais

Campo	Conteúdo
Identificador	Criação de Usuário/ Banco de Dados.
Função	Cadastrar o Usuário.
Descrição	Criar a identificação do usuário com nome, senha e idade válidos.
Entradas	Nome do usuário (caracteres e números), senha (caracteres, caracteres especiais e números) e idade (números).
Fonte	Usuário vai fornecer a informação através de um dispositivo de entrada.
Saídas	Usuário cadastrado e uma mensagem mostrando ao usuário que foi cadastrado.
Destino	O cadastro vai para o banco de dados e o usuário vai para a tela de login.
Ação	Os dados de entradas devem ser válidos seguindo as regras do programa (caracteres, complexidade e sem preenchimento vazio).
Requisitos	Um cadastro por usuário, e não deve ter usuários com mesmo nome(login) e a faixa etária válida conforme a regra.
Pré-condição	Complexidade da senha e sem preenchimento em vazio das entradas.
Pós-condições	Usuário estará cadastrado no sistema.
Efeitos colaterais	Mensagem de erro em caso de não cumprimento dos requisitos do programa.

5.2 – Apresentar a codificação da rotina

Apresente em pseudocódigo, Python ou outra linguagem

INÍCIO DA FUNÇÃO CADASTRAR USUÁRIO

REPETIR

SOLICITAR login ao usuário

SE o login estiver vazio ou contiver apenas espaços ENTÃO

EXIBIR "Erro: o login é obrigatório."

SENÃO SE o login possuir caracteres não permitidos ENTÃO

EXIBIR "Erro: o login deve conter apenas letras, números e sublinhado."

SENÃO

CONSULTAR o banco de dados para verificar se o login já está cadastrado

SE o login já existir ENTÃO

EXIBIR "Erro: este login já está cadastrado."

FIM SE

FIM SE

ATÉ que o login esteja preenchido, seja válido e não esteja cadastrado

REPETIR

SOLICITAR senha ao usuário

SE a senha estiver vazia ou contiver apenas espaços ENTÃO

EXIBIR "Erro: a senha é obrigatória."

SENÃO SE a senha possuir menos de 8 caracteres ENTÃO

EXIBIR "Erro: a senha deve possuir pelo menos 8 caracteres."

SENÃO SE a senha não possuir letra maiúscula OU não possuir letra minúscula OU não possuir número OU não possuir caractere especial

ENTÃO

EXIBIR "Erro: a senha deve conter letra maiúscula, letra minúscula, número e caractere especial."

SENÃO SE a senha possuir espaços ENTÃO

EXIBIR "Erro: a senha não pode conter espaços."

FIM SE

ATÉ que a senha cumpra todos os requisitos

REPETIR

SOLICITAR idade ao usuário

SE a idade estiver vazia ou contiver apenas espaços ENTÃO

EXIBIR "Erro: a idade é obrigatória."

SENÃO SE a idade não for um número inteiro ENTÃO

EXIBIR "Erro: informe uma idade numérica inteira."

SENÃO SE a idade for menor ou igual a zero ENTÃO

EXIBIR "Erro: a idade deve ser um número positivo."

FIM SE

ATÉ que a idade seja válida

TENTAR CADASTRAR no banco de dados:

login

senha

idade

SE o cadastro for realizado com sucesso ENTÃO

EXIBIR "Usuário cadastrado com sucesso."

SENÃO

EXIBIR "Erro: não foi possível realizar o cadastro."

FIM SE

FIM DA FUNÇÃO

6. ARQUITETURA DO SISTEMA

6.1. Visão Geral do Sistema

- **Nome do Sistema**: ARCOR QUEST
- **Objetivo do Sistema**: Oferecer uma experiência digital lúdica e educativa, no formato de quiz, que aproxime o usuário da marca Arcor e de seu portfólio (como Tortuguita e Triunfo), com perguntas, pistas e feedbacks baseados em material oficial da empresa.
- **Público-Alvo**: Jovens e adultos (adolescentes ao público geral), com linguagem, dificuldade e interface adequadas a esse perfil.
- **Principais Funcionalidades**: Cadastro e autenticação; tutorial interativo; seleção de categoria/marca e nível de dificuldade; respostas de múltipla escolha, verdadeiro/falso e associação; pontuação e feedback em tempo real; ranking de classificação.

6.2. Arquitetura Geral

Descrever o estilo arquitetural adotado (por exemplo, MVC, camadas, microsserviços, hexagonal, etc.):

Arquitetura em camadas (*Layered Architecture*), com separação entre apresentação, aplicação/API, domínio (regras de negócio) e persistência, comunicando-se por C# e MySql. Essa escolha atende diretamente ao RNFS04 (persistência confiável) e ao requisito obrigatório de banco de dados relacional (seção 3.1), pois isola a lógica de negócio do acesso a dados.

6.3. Componentes Arquiteturais e Camadas

6.3.1. Camada de Apresentação (Frontend)

- **Tecnologias**: C# e MySql
- **Responsabilidades**: Telas de cadastro, login, tutorial, quiz e ranking; validação inicial dos campos (ex.: formato do login antes de enviar)

6.3.2. Camada de Aplicação (Backend/API)

- **Tecnologias**: Node.js + Express

- **Responsabilidades**:autenticação e hash de senha; cálculo de pontuação e seleção de perguntas por categoria/dificuldade; exposição de endpoints REST — por exemplo POST /usuarios, POST /login, GET /perguntas, POST /respostas, GET /ranking

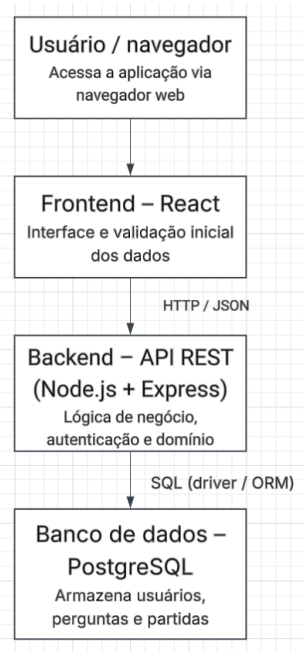
6.3.3. Camada de Domínio (Negócio)

- **Descrição**:
  - Contém as regras de negócio e os modelos de domínio
  - Independente das demais camadas
- **Componentes**:
  - Entidades: Usuario, Partida, Pergunta, Alternativa, Resposta, Marca (mesmas do diagrama de classe)
  - Casos de uso: Cadastrar e autenticar, Jogar quiz, Consultar ranking
  - Regras de validação: exatamente as do pseudocódigo da seção 5.2 (login único, senha com 8+ caracteres e caractere especial, idade numérica positiva)

6.3.4. Camada de Persistência (Banco de Dados)

- **Tecnologias**: MySql
- **Responsabilidades**:
  - armazenar usuários, perguntas, alternativas, marcas, partidas e respostas (RFS03, RNFS04); scripts de criação (schema.sql) e migrações

6.4. Diagrama da Arquitetura



6.5. Organização de Pastas e Módulos

```
/frontend
  /components   (LoginForm, TutorialStep, QuizCard, RankingList)
  /pages        (Login, Cadastro, Tutorial, Quiz, Ranking)
  /services     (api.js)
/backend
  /controllers  (usuarioController, perguntaController, partidaController, rankingController)
  /services     (quizService, pontuacaoService)
  /models       (Usuario, Pergunta, Alternativa, Marca, Partida, Resposta)
  /routes
  /config
/database
  schema.sql
  migrations/
```

6.6. Integrações Externas (se houver)

- O projeto não exige integrações externas obrigatórias — todo o conteúdo e a lógica de pontuação são internos (requisito da seção 1.2 de usar apenas material validado pela Arcor). A autenticação é própria (login/senha com hash, ex. bcrypt), sem OAuth2/Firebase. Como evolução futura, poderia ser adicionado compartilhamento da pontuação em redes sociais, citado na pesquisa Desk (item 2.2.1).

6.7. Ambiente e Implantação

- **Ambientes:** desenvolvimento (local), homologação (testes com a turma/professor), produção (apresentação final)
- **Contêineres:** Docker/Docker Compose (opcional, útil para padronizar backend + banco entre os integrantes)
- **CI/CD:** GitHub Actions (opcional, para rodar testes a cada push)
- **Hospedagem:** frontend no Vercel; backend no Render ou Railway; banco PostgreSQL gerenciado (ex.: Supabase ou ElephantSQL)

8. REFERÊNCIAS BIBLIOGRÁFICAS

SOMMERVILLE, I. **Engenharia de Software.** 11ª Edição. São Paulo: Pearson Addison-Wesley, 2017.